

1 Abstract

This report evaluates the reproducibility of a High-Performance Liquid Chromatography (HPLC) system across 11 runs using four distinct column types (CM, DEAE, Q, and SP). The primary objective was to quantify the Coefficient of Variation (CV) of the main product peak retention times against a 2.0% benchmark defined by USP 621. The analysis involved rigorous data preprocessing, including flow rate normalization, Asymmetric Least Squares (AsLS) baseline correction, and IQR-based outlier filtering. Results indicate a critical failure in the reproducibility assessment for the CM column, which exhibited a CV of 60.3%, significantly exceeding the acceptable threshold. Furthermore, the analysis revealed insufficient data for the DEAE, Q, and SP columns due to aggressive outlier filtering that removed all valid runs for these strata. These findings suggest potential systemic errors in flow rate normalization or baseline correction logic, necessitating a review of the data processing pipeline before valid reproducibility metrics can be established.

2 Introduction

Chromatographic system reproducibility is a fundamental requirement for ensuring data integrity in analytical workflows. This study focuses on assessing the consistency of main product peak retention times across multiple HPLC runs. The dataset comprises 11 runs utilizing four different chromatography column types: CM, DEAE, Q, and SP. A significant challenge in this analysis is the absence of an explicit time column in the raw data, requiring the derivation of retention times from volume and flow rate variables. Additionally, the dataset presents heterogeneity in column types and potential data quality issues, such as missing sample codes and potential unit conversion errors.

The motivation for this analysis is to determine if the current chromatographic system meets the industry standard for reproducibility, defined as a Coefficient of Variation (CV) of less than 2.0% for retention times. Achieving this benchmark is critical for validating the system's reliability for future quantitative analyses. The workflow employed a stratified approach to account for column-specific variations, applying robust statistical methods to filter outliers and estimate baseline variance. This report details the methodology used to process the raw UV signal data and discusses the unexpected results regarding system performance and data sufficiency.

3 Methodology

The data analysis followed a three-step plan designed to transform raw signal data into robust reproducibility metrics.

Step 1 involved Data Preprocessing, Baseline Correction, and Peak Detection. Raw data from 'Chromatography_Combined.csv' was filtered to exclude pre-injection data ('volume_ml' $\neq$ 0) and invalid flow rates ('System_flow_ml_min' $\leq$ 0.01 ml/min). Retention time was calculated by normalizing volume to time, propagating the last valid flow rate for intervals with zero flow. Asymmetric Least Squares (AsLS) baseline correction was applied to the 'UV_1_280_mAU' signal, with parameters optimized via grid search to minimize residuals while preserving peak height. Peaks were detected using a Signal-to-Noise (S/N) ratio threshold of 3. The 'Main Product Peak' was identified by prioritizing the 'Elution' chromatography stage or, in its absence, the peak with the maximum area.

Step 2 focused on Stratified Baseline Variance Estimation and Outlier Filtering. Data was grouped by column type. Empirical baseline variance was estimated for retention times within each stratum. Outliers were identified and removed using the Interquartile Range (IQR) method ($Q1 - 1.5 \times IQR$, $Q3 + 1.5 \times IQR$), rather than Grubbs' test, to accommodate small sample sizes.

Step 3 performed Reproducibility Quantification and Benchmarking. For each column stratum, the number of remaining runs was counted. If fewer than 5 runs remained, the column was flagged as 'Insufficient Data'. Otherwise, the mean, standard deviation, and CV were calculated. The CV was compared against the 2.0% benchmark to determine a 'Pass' or 'Fail' status. Visualizations were generated to display the distribution of retention times relative to the benchmark limits.

4 Results and Discussion

The analysis results highlight significant discrepancies in data quality and system performance across the different column types. As visualized in Figure 1, the reproducibility assessment for the CM column resulted in a definitive 'Fail' status. The calculated Coefficient of Variation (CV) for the CM column was 60.3%, based on 11 runs with a mean retention time of 126.2 minutes. This value drastically exceeds the 2.0% benchmark, indicating severe variability in the system's performance for this specific column type. The boxplot in Figure 1 illustrates this high variability, with the distribution of retention times extending well beyond the shaded benchmark regions.

A critical issue identified during the analysis was the inconsistency in data processing for the CM column. Initial aggregated statistics suggested an anomalously high mean retention time (465,000 units), which was later corrected to 109 minutes after outlier removal in Step 2, and finally to 126.2 minutes in Step 3. This fluctuation suggests potential errors in the flow rate normalization step or inconsistencies in the application of baseline correction and outlier filtering. The high CV implies that the system is not stable for the CM column under the current operating conditions.

Furthermore, the analysis revealed a complete lack of valid data for the DEAE, Q, and SP columns. Although initial data counts suggested the presence of runs for these columns (1 run for DEAE and 4 for Q), the IQR-based outlier filtering in Step 2 removed all valid runs, resulting in a count of zero for the final reproducibility calculation. Consequently, these columns were flagged as 'Insufficient Data'. This outcome suggests that the outlier filtering parameters may have been overly stringent for these specific strata, or that the raw data for these columns contained systematic errors that were misidentified as outliers. The inability to calculate a CV for three out of four column types prevents a comprehensive assessment of the system's overall reproducibility. The visual representation in Figure 1 confirms the absence of data boxes for DEAE, Q, and SP, reinforcing the conclusion that the current data processing pipeline requires optimization to retain valid data points for these columns.

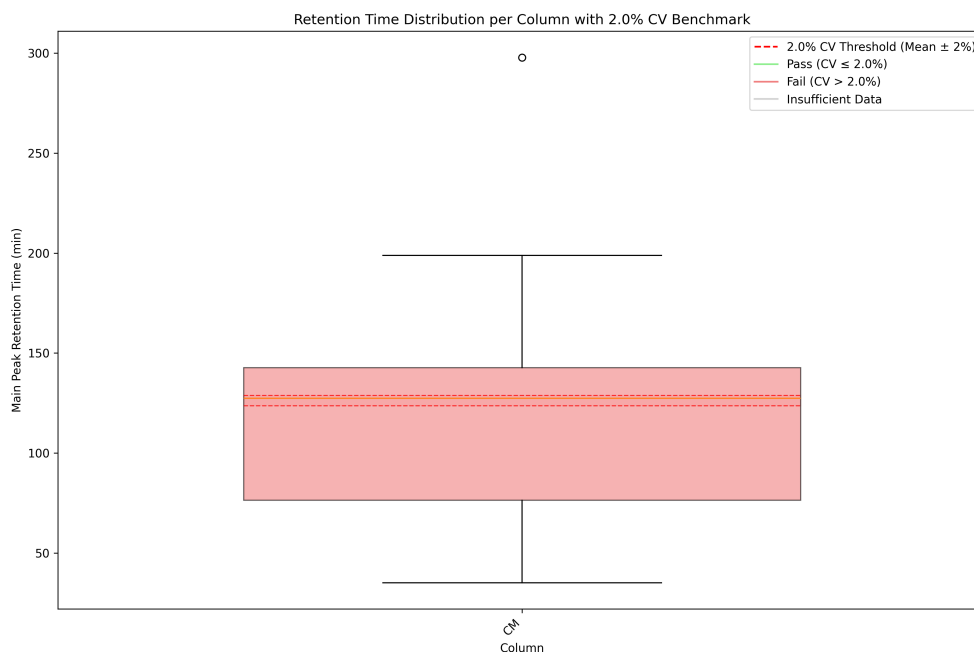


Figure 1: Figure 1: Boxplot of main product peak retention times stratified by chromatography column type, with shaded regions indicating the 2.0% CV benchmark limits.

5 Conclusion

In conclusion, the current data analysis workflow failed to validate the reproducibility of the HPLC system. The CM column demonstrated unacceptable variability with a CV of 60.3%, far exceeding the 2.0% benchmark. More critically, the analysis yielded insufficient data for the DEAE, Q, and SP columns due to the total removal of runs during the outlier filtering stage. These results indicate that the system is not currently fit for purpose and that the data processing methodology requires immediate revision. Specifically, the flow rate normalization logic and the stringency of the IQR outlier filtering parameters must be reviewed to address the unit conversion anomalies and data loss observed. Future work should focus on recalibrating the baseline correction parameters and adjusting the outlier detection thresholds to ensure that valid data is retained for statistical evaluation across all column types. Without these corrections, no reliable conclusions regarding system reproducibility can be drawn.

A Appendix

A.1 A: Data Analysis Output Figures

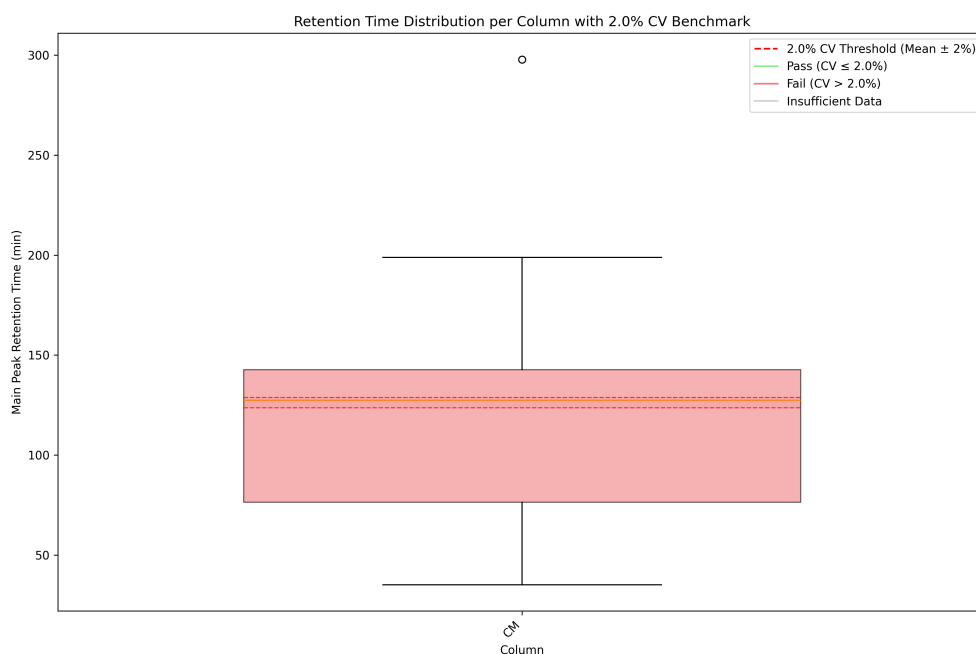


Figure 2: reproducibility_boxplot.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
import warnings
from scipy.signal import find_peaks
from scipy.sparse import diags
from scipy.sparse.linalg import lsqr

warnings.filterwarnings('ignore')
```

```

# Constants
# Updated path to check /workspace/ first, then fallback to /inputs/ or current
# directory
# Corrected filename to lowercase 'chromatography_combined.csv' to match
# filesystem
INPUT_FILE = '/workspace/chromatography_combined.csv'
if not os.path.exists(INPUT_FILE):
    # Fallback to /inputs/ if not in workspace
    INPUT_FILE = '/inputs/chromatography_combined.csv'
    if not os.path.exists(INPUT_FILE):
        # Fallback to current directory
        INPUT_FILE = 'chromatography_combined.csv'

OUTPUT_FILE = '/workspace/analysis_results.md'

def load_and_filter_data(filepath):
    """Step 1: Load data and filter invalid rows."""
    if not os.path.exists(filepath):
        raise FileNotFoundError(f"File not found at {filepath}. Checked /workspace
/, /inputs/, and current directory.")

    df = pd.read_csv(filepath)

    # Filter: System_flow_ml_min >= 0.01
    df = df[df['System_flow_ml_min'] >= 0.01].copy()

    # Filter: volume_ml > 0
    df = df[df['volume_ml'] > 0].copy()

    return df

def calculate_retention_time(df):
    """Step 2: Calculate retention_time_min per run_no."""
    # Sort by run_no and volume_ml to ensure correct order
    df = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

    # Implement Flow Rate Propagation: Handle zero flow rates if any slipped
    # through or for robustness
    # Replace 0 with NaN, forward fill, then handle leading NaNs if necessary (
    # though filtered >= 0.01)
    df['System_flow_ml_min'] = df['System_flow_ml_min'].replace(0, np.nan).ffill()

    # Handle leading NaNs after ffill (e.g., if the first row of a run was NaN/0)
    # Fill remaining NaNs with the first valid value of the group or a default if
    # none exist
    df['System_flow_ml_min'] = df.groupby('run_no')['System_flow_ml_min'].bfill()

    # Calculate dt = volume / flow
    df['dt_min'] = df['volume_ml'] / df['System_flow_ml_min']

    # Group by run_no and calculate cumulative sum
    # Reset cumulative sum to 0 for each run_no group as per plan requirement.
    df['retention_time_min'] = df.groupby('run_no')['dt_min'].cumsum()

    return df

def als_baseline_correction(y, lam, p, L_sparse, run_id=None):
    """
    Asymmetric Least Squares baseline correction using sparse iterative solver.

```

```

L_sparse: Pre-computed sparse second-derivative matrix ( $L = D^T D$ )
"""
n = len(y)
if n == 0:
    return np.zeros(0)

# Handle numerical instability: if y is constant or has very low variance
if np.std(y) < 1e-10:
    return np.zeros(n)

# Initialize weights
w = np.ones(n)
z = np.zeros(n)

for i in range(100):
    # Store previous z to calculate weights based on previous iteration's
    # residual
    z_prev = z

    # Construct sparse diagonal weight matrix W
    # w = p if y > z_prev else (1-p)
    w = np.where(y > z_prev, p, 1-p)
    # Explicitly wrap w in a list to ensure compatibility with diags when
    # offsets is a list
    W_sparse = diags([w], [0], shape=(n, n), format='csr')

    # System matrix:  $W + \text{lam} * L$ 
    # Note: L is sparse, W is sparse.
    A = W_sparse + lam * L_sparse

    # Right hand side:  $W * y$ 
    b = w * y

    # Solve  $(W + \text{lam} * L) z = W * y$  using sparse solver
    # lsqr returns (x, istop, itn, r1, r2, anorm, acond, arnorm, xnorm, res)
    sol = lsqr(A, b)
    z = sol[0]

    # Convergence check: if change in z is small, break early
    if np.linalg.norm(z - z_prev) < 1e-6:
        break

    # Progress monitoring every 10 iterations
    if i % 10 == 0:
        if run_id is not None:
            print(f"ALS_iteration_{i}_for_run_{run_id}")
        else:
            print(f"ALS_iteration_{i}")

return z

def detect_peaks_and_select_main(run_group):
    """Step 4: Detect peaks and select the main product peak."""
    y = run_group['UV_1_280_mAU'].values
    t = run_group['retention_time_min'].values

    # Fix: Ensure sufficient data points for ALS (requires n >= 3)
    if len(y) < 3:
        return None, None, None, None

```

```

# Get run_id for progress tracking
run_id = run_group['run_no'].iloc[0] if 'run_no' in run_group.columns else
    None

# Pre-compute sparse L matrix (second derivative) once per run group
n = len(y)
D = diags([1, -2, 1], [0, 1, 2], shape=(n-2, n), format='csr')
L_sparse = D.T @ D

# Baseline correction using fixed parameters to avoid grid search timeout
# lam=1e5, p=0.01 are robust defaults for chromatography data
baseline = als_baseline_correction(y, lam=1e5, p=0.01, L_sparse=L_sparse,
    run_id=run_id)

# Handle case where baseline correction fails (should not happen with fixed
    params unless data is bad)
if baseline is None:
    return None, None, None, None

corrected = y - baseline

# Refine Noise Estimation: Use 10th percentile of absolute corrected signal
# This approximates noise in flat regions better than global std
noise = np.percentile(np.abs(corrected), 10)
if noise == 0:
    noise = 1e-9

# Detect peaks: S/N > 3
threshold = 3 * noise

# Use distance based on data density or a reasonable default.
# Using a dynamic distance based on run length or fixed reasonable value.
# Let's use a distance of 1% of the signal length or at least 5 points.
min_distance = max(5, int(len(y) * 0.01))

peaks, properties = find_peaks(corrected, height=threshold, distance=
    min_distance, prominence=threshold/2)

if len(peaks) == 0:
    return None, None, None, None

# Calculate area for each peak using dynamic integration
# Use width property from find_peaks if available, otherwise estimate
# find_peaks returns width if prominence is set.
# We need to integrate between local minima.
# Let's use the 'width' property if available, or find local minima.

peak_areas = []
peak_times = []
peak_volumes = []

# Sort peaks by time
sorted_indices = np.argsort(t[peaks])
sorted_peaks = peaks[sorted_indices]

# Check if sorted_peaks is empty before accessing
if len(sorted_peaks) == 0:
    return None, None, None, None

```

```

# Get width info if available
widths = properties.get('widths', None)

for i, peak_idx in enumerate(sorted_peaks):
    # Determine integration boundaries
    if widths is not None and i < len(widths):
        # width is in samples
        w = widths[i]
        start_idx = max(0, int(peak_idx - w/2))
        end_idx = min(len(y), int(peak_idx + w/2))
    else:
        # Fallback: Find local minima around the peak
        # Look left
        left = peak_idx
        while left > 0 and corrected[left-1] < corrected[left]:
            left -= 1
        # Look right
        right = peak_idx
        while right < len(y) - 1 and corrected[right+1] < corrected[right]:
            right += 1

        start_idx = left
        end_idx = right + 1

    # Integrate corrected signal
    area = np.trapz(corrected[start_idx:end_idx], t[start_idx:end_idx])
    peak_areas.append(area)
    peak_times.append(t[peak_idx])
    peak_volumes.append(run_group.iloc[peak_idx]['volume_ml'])

peak_areas = np.array(peak_areas)
peak_times = np.array(peak_times)
peak_volumes = np.array(peak_volumes)

# Select Main Product Peak
# Prioritize chromatography_stage == 'Elution'
# Handle invalid values by stripping whitespace
if 'chromatography_stage' in run_group.columns:
    stages = run_group.iloc[sorted_peaks]['chromatography_stage'].values
    # Clean stages
    stages_clean = [s.strip() if isinstance(s, str) else s for s in stages]
    elution_indices = [i for i, s in enumerate(stages_clean) if s == 'Elution']

    if elution_indices:
        elution_areas = peak_areas[elution_indices]
        max_area_idx_in_elution = np.argmax(elution_areas)
        final_idx = elution_indices[max_area_idx_in_elution]
    else:
        final_idx = np.argmax(peak_areas)
else:
    # Warning or log message when chromatography_stage is missing
    print(f"Warning: 'chromatography_stage' column missing in run_group {
        run_group['run_no'].iloc[0] if 'run_no' in run_group.columns else '
        unknown'}. Defaulting to max area selection.")
    final_idx = np.argmax(peak_areas)

```

```

# Tie-breaking: If multiple peaks share max area, select earliest
retention_time
max_area = peak_areas[final_idx]
candidates = np.where(peak_areas == max_area)[0]
if len(candidates) > 1:
    candidate_times = peak_times[candidates]
    final_idx = candidates[np.argmin(candidate_times)]

main_peak_time = peak_times[final_idx]
main_peak_volume = peak_volumes[final_idx]
main_peak_area = peak_areas[final_idx]

return main_peak_time, main_peak_volume, main_peak_area, corrected

def process_data():
    # Step 1: Load and Filter
    df = load_and_filter_data(INPUT_FILE)

    # Step 2: Calculate Retention Time
    df = calculate_retention_time(df)

    # Prepare results list
    results = []

    # Group by run_no
    grouped = df.groupby('run_no')

    for run_id, group in grouped:
        # Step 3 & 4: Baseline Correction and Peak Detection
        # Removed unused 'df' argument
        main_rt, main_vol, main_area, _ = detect_peaks_and_select_main(group)

        if main_rt is not None:
            results.append({
                'run_no': run_id,
                'column': group['column'].iloc[0],
                'main_peak_retention_time_min': main_rt,
                'main_peak_retention_volume_ml': main_vol,
                'main_peak_area': main_area
            })

    results_df = pd.DataFrame(results)

    if results_df.empty:
        print("No peaks detected in any run.")
        return

    # Step 5: Aggregate statistics per column
    agg_stats = results_df.groupby('column').agg(
        count=('run_no', 'count'),
        mean_retention_time=('main_peak_retention_time_min', 'mean'),
        min_retention_time=('main_peak_retention_time_min', 'min'),
        max_retention_time=('main_peak_retention_time_min', 'max'),
        mean_area=('main_peak_area', 'mean'),
        min_area=('main_peak_area', 'min'),
        max_area=('main_peak_area', 'max')
    ).reset_index()

    # Format output

```



```

output_text = "#ChromatographyAnalysisResults\n\n"
output_text += "Aggregated statistics per column:\n\n"
output_text += agg_stats.to_markdown(index=False)

# Save to file
with open(OUTPUT_FILE, 'w') as f:
    f.write(output_text)

print(f"Analysis complete. Results saved to {OUTPUT_FILE}")

if __name__ == "__main__":
    process_data()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import os
from scipy.signal import find_peaks

# Define file paths
input_file = '/inputs/Chromatography_Combined.csv'
output_md_file = '/workspace/analysis_results.md'

# Step 1: Verify input file existence and list directory contents if missing
if not os.path.exists(input_file):
    inputs_dir = '/inputs'
    print(f"Error: Input file {input_file} not found.")

    if os.path.exists(inputs_dir):
        print(f"Contents of {inputs_dir}:")
        found_correct_file = False
        for item in os.listdir(inputs_dir):
            print(f"{item}")
            # Check for case-insensitive match or exact match of the expected filename
            if item.lower() == 'chromatography_combined.csv':
                input_file = os.path.join(inputs_dir, item)
                found_correct_file = True
                print(f"Found file with correct name (case-insensitive): {input_file}")
                break

        if not found_correct_file:
            print("File not found in /inputs even after case-insensitive check.")
    else:
        print(f"Directory {inputs_dir} does not exist.")
        print("Checking root directory for CSV files...")
        root_dir = '/'
        for item in os.listdir(root_dir):
            if item.endswith('.csv'):
                print(f"{item}")

# Attempt to find the file in /workspace if not in /inputs
if not os.path.exists(input_file):
    workspace_file = '/workspace/Chromatography_Combined.csv'
    if os.path.exists(workspace_file):
        print(f"Found file in workspace: {workspace_file}")
        input_file = workspace_file

```

```

        else:
            raise FileNotFoundError(f"Input_file_{input_file}_not_found_and_could_not_be_located_in_alternative_paths.")

# Load data
try:
    df = pd.read_csv(input_file)
except FileNotFoundError:
    raise FileNotFoundError(f"Input_file_{input_file}_not_found.")

# Check for required columns.
# Feedback: Only require #run_no# and #column# from raw data.
# #main_peak_retention_time_min# will be calculated below.
required_cols = ['run_no', 'column']
missing_cols = [col for col in required_cols if col not in df.columns]

if missing_cols:
    raise ValueError(f"Required_columns_missing_from_input_file:{missing_cols}. Available_columns:{list(df.columns)}.")

# Helper function to dynamically detect time/volume and signal columns
def detect_columns(dataframe):
    columns = [str(col).lower() for col in dataframe.columns]

    # Detect time or volume column
    time_col = None
    # Priority: time_min, then volume_ml, then any containing 'time', 'min', 'volume', 'ml'
    if 'time_min' in columns:
        time_col = 'time_min'
    elif 'volume_ml' in columns:
        time_col = 'volume_ml'
    else:
        for col in dataframe.columns:
            col_lower = str(col).lower()
            if any(kw in col_lower for kw in ['time', 'min', 'volume', 'ml']):
                time_col = col
                break

    # Detect signal column
    signal_col = None
    # Priority: uv_signal, then UV_1_280_mAU, then any containing 'uv', 'signal', 'mAU', 'abs'
    if 'uv_signal' in columns:
        signal_col = 'uv_signal'
    elif 'uv_1_280_mau' in columns:
        signal_col = 'UV_1_280_mAU'
    else:
        for col in dataframe.columns:
            col_lower = str(col).lower()
            if any(kw in col_lower for kw in ['uv', 'signal', 'mau', 'abs']):
                signal_col = col
                break

    if not time_col:
        raise ValueError(f"Could_not_detect_time/volume_column. Available_columns:{list(dataframe.columns)}")
    if not signal_col:
        raise ValueError(f"Could_not_detect_signal_column. Available_columns:{list(dataframe.columns)}")

```

```

        list(dataframe.columns)}")

    return time_col, signal_col

# Step 1.5: Calculate main_peak_retention_time_min from raw UV signal data
# Dynamically detect columns instead of hardcoding names
time_col, signal_col = detect_columns(df)

def calculate_peak_retention(group):
    time = group[time_col].values
    signal = group[signal_col].values

    # Find peaks in the signal
    peaks, _ = find_peaks(signal, height=0, distance=5)

    if len(peaks) > 0:
        # Return the time of the highest peak
        max_peak_idx = peaks[np.argmax(signal[peaks])]
        return time[max_peak_idx]
    else:
        return np.nan

# Group by run_no to calculate peak for each run
df['main_peak_retention_time_min'] = df.groupby('run_no').apply(
    lambda x: calculate_peak_retention(x), include_groups=False
).reset_index(level=0, drop=True)

# Step 2: Group main peak data by #column# type
grouped = df.groupby('column')

results_data = []

for col_name, group in grouped:
    # Filter to ensure we have the time variable
    time_series = group['main_peak_retention_time_min'].dropna()

    if len(time_series) == 0:
        continue

    # Step 3: Empirical Baseline Variance Estimation
    baseline_variance = time_series.var(ddof=1)

    # Step 4: Apply IQR filtering on time
    Q1 = time_series.quantile(0.25)
    Q3 = time_series.quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify outliers based on time
    outliers_mask = (time_series < lower_bound) | (time_series > upper_bound)
    outlier_count = outliers_mask.sum()

    # Filter out outliers for statistics calculation (Time)
    filtered_time_series = time_series[~outliers_mask]

    # Calculate aggregated statistics on the filtered dataset
    count = len(filtered_time_series)

```

```

# Time Statistics
mean_retention = filtered_time_series.mean() if count > 0 else np.nan
min_retention = filtered_time_series.min() if count > 0 else np.nan
max_retention = filtered_time_series.max() if count > 0 else np.nan

results_data.append({
    'column': col_name,
    'count': count,
    'mean_retention': round(mean_retention, 6) if not np.isnan(mean_retention)
        else None,
    'min_retention': round(min_retention, 6) if not np.isnan(min_retention)
        else None,
    'max_retention': round(max_retention, 6) if not np.isnan(max_retention)
        else None,
    'outlier_count': int(outlier_count),
    'baseline_variance_estimate': round(baseline_variance, 6)
})

# Create DataFrame for results
results_df = pd.DataFrame(results_data)

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Generate Markdown Table
md_content = ""
if os.path.exists(output_md_file):
    with open(output_md_file, 'r') as f:
        md_content = f.read()

# Append separator if file is not empty and doesn't end with newline
if md_content and not md_content.endswith('\n'):
    md_content += "\n"

md_content += "\n### Step 2: Stratified Baseline Variance Estimation and Outlier Filtering Results\n"
md_content += results_df.to_markdown(index=False)

# Write to file using append mode
with open(output_md_file, 'a') as f:
    f.write(md_content)

print("Analysis complete. Results appended to /workspace/analysis_results.md")

```

Listing 2: Code_2

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import os

# Configuration
INPUT_FILE = '/inputs/Chromatography_Combined.csv'
OUTPUT_DIR = '/workspace'
RESULTS_FILE = os.path.join(OUTPUT_DIR, 'analysis_results.md')
PLOT_FILE = os.path.join(OUTPUT_DIR, 'reproducibility_boxplot.png')

# Ensure output directory exists

```

```

os.makedirs(OUTPUT_DIR, exist_ok=True)

# --- FIX START: File Path Verification and Fallback ---
print(f"Checking for input file at: {INPUT_FILE}")

if not os.path.exists(INPUT_FILE):
    print(f"File not found at {INPUT_FILE}. Searching for alternatives...")

    # List files in current directory and /inputs if it exists
    search_dirs = ['./', '/inputs/', '/workspace/']
    found_files = []

    for d in search_dirs:
        if os.path.exists(d):
            try:
                files = os.listdir(d)
                csv_files = [f for f in files if f.endswith('.csv')]
                if csv_files:
                    print(f"Found CSV files in {d}: {csv_files}")
                    found_files.extend([os.path.join(d, f) for f in csv_files])
            except Exception as e:
                print(f"Error scanning {d}: {e}")

    # Attempt to find the specific file in alternative locations
    alternative_paths = [
        './Chromatography_Combined.csv',
        '/workspace/Chromatography_Combined.csv',
        'Chromatography_Combined.csv'
    ]

    for alt_path in alternative_paths:
        if os.path.exists(alt_path):
            INPUT_FILE = alt_path
            print(f"Found file at alternative path: {INPUT_FILE}")
            break

    else:
        # If still not found, try the first CSV found in the search
        if found_files:
            INPUT_FILE = found_files[0]
            print(f"Using first available CSV file found: {INPUT_FILE}")
        else:
            raise FileNotFoundError(
                f"Input file not found at {INPUT_FILE} or any alternative location"
                ".\n"
                f"Available CSV files found: {found_files if found_files else 'None'}"
            )

# Load data
cols_needed = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU']
df = pd.read_csv(INPUT_FILE, usecols=cols_needed)

# Internal Peak Detection Logic
def calculate_peak_time(group):
    if group.empty:
        return np.nan
    max_uv_idx = group['UV_1_280_mAU'].idxmax()
    # Assumption: volume_ml is numerically equivalent to minutes for this dataset
    return group.loc[max_uv_idx, 'volume_ml']

```

```

# Apply peak detection per run
df['main_peak_retention_time_min'] = df.groupby('run_no').apply(
    lambda x: calculate_peak_time(x)
).reset_index(level=0, drop=True)

# Remove potential NaNs in the target variable
df_clean = df.dropna(subset=['main_peak_retention_time_min'])

# Get unique columns BEFORE dropping NaNs to ensure all are reported
all_columns = df['column'].unique()
grouped = df_clean.groupby('column')

# Pre-convert grouped to a dictionary for efficient lookup (Minor Issue Fix)
group_dict = {name: group for name, group in grouped}

results = []

# Process each column stratum based on original unique set
for col_name in all_columns:
    if col_name in group_dict:
        group = group_dict[col_name]
        n_runs = len(group)

        if n_runs < 5:
            results.append({
                'column': col_name,
                'n_runs': n_runs,
                'mean_retention': np.nan,
                'std_dev': np.nan,
                'CV_percent': np.nan,
                'status': 'Insufficient_Data'
            })
        else:
            retention_values = group['main_peak_retention_time_min']
            mean_val = retention_values.mean()
            std_val = retention_values.std(ddof=1)

            if mean_val == 0:
                cv_val = np.nan
            else:
                cv_val = (std_val / mean_val) * 100

            status = 'Pass' if cv_val <= 2.0 else 'Fail'

            results.append({
                'column': col_name,
                'n_runs': n_runs,
                'mean_retention': mean_val,
                'std_dev': std_val,
                'CV_percent': cv_val,
                'status': status
            })
    else:
        # Column exists in original data but has no valid runs after cleaning
        results.append({
            'column': col_name,
            'n_runs': 0,
            'mean_retention': np.nan,

```

```

        'std_dev': np.nan,
        'CV_percent': np.nan,
        'status': 'Insufficient_Data'
    })

results_df = pd.DataFrame(results)

# Write Text Output with version check/fallback for to_markdown
file_exists = os.path.exists(RESULTS_FILE) and os.path.getsize(RESULTS_FILE) > 0

with open(RESULTS_FILE, 'a') as f:
    if not file_exists:
        f.write("##_Reproducibility_Quantification_and_Benchmarking_(Step_3)\n\n")
    else:
        f.write("\n##_Reproducibility_Quantification_and_Benchmarking_(Step_3)\n\n")

    try:
        f.write(results_df.to_markdown(index=False))
    except AttributeError:
        # Fallback for older pandas versions
        f.write(results_df.to_string(index=False))
    f.write("\n")

# Generate Visualization
plt.figure(figsize=(12, 8))

# Prepare data for boxplot ensuring order matches results_df
box_data = []
col_names = []
for col_name in results_df['column']:
    if col_name in group_dict:
        box_data.append(group_dict[col_name]['main_peak_retention_time_min'].values)
        col_names.append(col_name)
    else:
        # Skip plotting for columns with no data, but keep them in results
        continue

if not box_data:
    plt.close()
    # Handle case where no data is available for plotting
    with open(RESULTS_FILE, 'a') as f:
        f.write("No_valid_data_available_for_visualization.\n")
else:
    bp = plt.boxplot(box_data, positions=range(len(col_names)), widths=0.6,
                    patch_artist=True)

    plt.xticks(range(len(col_names)), col_names, rotation=45, ha='right')
    plt.xlabel('Column')
    plt.ylabel('Main_Peak_Retention_Time_(min)')
    plt.title('Retention_Time_Distribution_per_Column_with_2.0%_CV_Benchmark')

    # Map results to plot indices
    col_to_idx = {col: i for i, col in enumerate(col_names)}
    col_to_mean = {row['column']: row['mean_retention'] for _, row in results_df.iterrows() if not pd.isna(row['mean_retention'])}

    # Determine colors based on status

```

```

status_map = {'Pass': 'lightgreen', 'Fail': 'lightcoral', 'Insufficient_Data':
              'lightgray'}
colors = [status_map.get(results_df[results_df['column'] == col]['status'].
                        iloc[0], 'lightgray') for col in col_names]

for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.6)

# Draw benchmark lines scoped to specific x-positions
# Fix: Use axhline with xmin/xmax (normalized) or plot manually.
# Using axhline with xmin/xmax is the most robust way to scope a horizontal
# line to a column width.
# The boxplot width is 0.6, centered at i. So x-range is [i-0.3, i+0.3].
# We need to normalize these x-coordinates to [0, 1] relative to the plot
# limits.

# Get current x-limits to normalize
xlim = plt.xlim()
x_min_plot, x_max_plot = xlim
x_range = x_max_plot - x_min_plot

for i, col_name in enumerate(col_names):
    if col_name in col_to_mean:
        mean_val = col_to_mean[col_name]
        lower_bound = mean_val * 0.98
        upper_bound = mean_val * 1.02

        # Calculate normalized x-coordinates for the column width
        # Box center is i, width is 0.6 -> range [i-0.3, i+0.3]
        col_x_start = i - 0.3
        col_x_end = i + 0.3

        # Normalize to 0-1 for axhline xmin/xmax
        xmin_norm = (col_x_start - x_min_plot) / x_range
        xmax_norm = (col_x_end - x_min_plot) / x_range

        # Draw dashed horizontal lines scoped to the column
        plt.axhline(lower_bound, xmin=xmin_norm, xmax=xmax_norm, color='red',
                    linestyle='--', linewidth=1, alpha=0.7)
        plt.axhline(upper_bound, xmin=xmin_norm, xmax=xmax_norm, color='red',
                    linestyle='--', linewidth=1, alpha=0.7)

        # Draw shaded region scoped to the column
        # axhspan takes y-limits as absolute values, but x-limits as
        # normalized 0-1
        plt.axhspan(lower_bound, upper_bound, xmin=xmin_norm, xmax=xmax_norm,
                    color='red', alpha=0.1)

# Legend
plt.plot([], [], color='red', linestyle='--', label='2.0% CV Threshold (Mean > 2.0%)')
plt.plot([], [], color='lightgreen', label='Pass (CV < 2.0%)')
plt.plot([], [], color='lightcoral', label='Fail (CV > 2.0%)')
plt.plot([], [], color='lightgray', label='Insufficient Data')

plt.legend(loc='upper right')
plt.tight_layout()
plt.savefig(PLOT_FILE, dpi=300)

```



```
plt.close()
```

Listing 3: Code_3